



Ch03-4 一些常用包与类

余 烨

Java的常用包

- Java.**lang**：包括一些Java语言的基本类和核心类，如String、Math、Integer、System和Runtime，提供常用的功能，这个包中的所有类是被隐式导入的
- Java.**io**：包含一些用作输入输出（I/O）处理的类
- Java.**net**：包含用于建立网络连接的类，与Java.io同时使用完成于网络有关的读写
- Java.**util**：包含一些实用工具类和数据结构类

常用类

- java.lang 包中提供了大量的常用类
  - String类
  - 基本数据类型的包装类（Integer、Float等）
  - 枚举类（Enum）
  - 基本数学运算（Math）
  - 线程（Thread）
  - ……
  - 与以上类运行相关的异常和错误处理

1. 字符串

- 字符串是**字符组成的序列**，是编程常用的数据类型。字符串分为常量和变量。
  - 字符串常量是指其值保持不变的量，使用双引号括起来的字符序列。
  - 字符串变量要通过String 类来实现，需要声明和初始化。

声明格式： String 字符串变量；

初始化： 字符串变量 = new String（）；

合并使用 String 字符串变量=new String（）；

```
S1=“ wang san ”;
S2=“is a student”;
System.out.println(s1+s2);
```

运算符‘+’的作用是将前后两个字符串连接起来

String类

- 针对String的“+”和“+=”，是Java中唯一被重载的操作符；在Java中，不允许程序员重载操作符。
- 例如：
  - String str = "abc";
  - int i = 3;
  - float a = 1.5f;
  - char ch = 'f';
  - boolean b = false;
  - System.out.println(str + i + a + ch + b);

字符串操作

- 访问字符串对象

1. length()　返回字符串长度
2. char charAt(int index)
   返回字符串中第index个字符
3. int indexOf(int ch)
   返回字符串中字符ch第一次出现的位置
4. int indexOf(String str, int index)
   返回值为，从字符串的第index位置开始，子串str第一次出现的位置
5. subString(int index1, int index2)
   返回的是从字符串的第index1位置开始到index2位置结束的子串

操作示例：

String s=new String();

s="I am a student.";

s.length()　?

s.charAt(7)　?

s.indexOf('a')　?

注意字符串从0计数

找不到其值为-1

s.indexOf("stu",0)　?

s.indexOf("stu",9)　?

s.subString(7,13)　?

字符串比较

- 字符在计算机中是按照Unicode编码存储的，两个字符串的比较实际上是字符串中对应字符编码的比较。从首字符开始逐个向后比较对应字符，当发现了一对不同的字符或到字符串末尾，两个字符串比较结束。

常用的成员方法

1. equals (Object obj)
   本字符串与obj字符串比较，相等返回true, 不等返回false
2. equalsIgnoreCase(String str)字符串比较，忽略大小写
3. compareTo(String str)返回值为整型，本字符串大于str取正值，小于取负值，相等取0值

示例

String s=new String("student");

s.equals("Student") ?

s.equalsIgnoreCase("Student") ?

s.compareTo("five student") ?

s.compareTo("two student") ?

字符串与其他类型的转换

方　　法	返回值类型	返回值
Boolean.getBoolean(“false”)	boolean	false
Integer.parseInt(“123”)	int	123
Long.parseLong(“375”)	long	375
Float.parseFloat(“345.23”)	float	345.23
Double.parseDouble(“67892.34”)	double	67892.34

- 基本类型数据转换成字符串的方法是
  - String.valueOf(基本数据类型)
  - 如 String.valueOf (123)的值为 “123”
  - String.valueOf (0.34)的值为 “0.34”

字符串与字符数组

- 将字符数组转换为字符串
  - String(char[]);
  - String(char[], int offset, int length);

```
String str = new String( “I Love Java");
char[] charArray = new char[6];
str.getChars(4, 8, charArray, 0);
System.out.println(charArray );
```

- 将字符串转换为字符数组
  - String类中

public void getChars(int start, int end, char c[], int offset);

将当前字符串中从start到end-1的字符复制到c中，在c中从offset位置开始存放

public char[] toCharArray();

将当前字符串转换为子节数组。

```
String Str1 = “Hello World”;
char[] charArray = str.toCharArray();
System.out.println(charArray);
```

字符串与字节数组

- 将字节数组转化为字符串
  - String(byte[]);
  - String(byte[], int offset, int length);
- 将字符串转化为字节数组
  - String类中

public byte[] getBytes();

将当前字符串转换为字节数组

```
String str = new String( “I Love Java");
byte[] byteArray = str1.getBytes();
System.out.println(byteArray );
```

public byte[] getBytes(String charsetName);

使用参数指定的字符编码，将当前字符串转换为一个字节数组。

String类其他方法

- public Boolean startsWith(String s);
- public Boolean endsWith(Sting s);

判断当前字符串是否以s开头或结尾。

- public boolean regionMatches(int firstStart, Sting other, int otherStart, int length)

从当前字符串参数firstStart指定位置开始，取长度为length的子串，并将这个子串和other的从otherStart位置开始，长度为length的子串相比，如果相同则返回true。

- public boolean contains(String s)

判断当前字符串对象是否含有s子串。

- public String trim()

获得当前字符串去掉前后空格后的字符串。

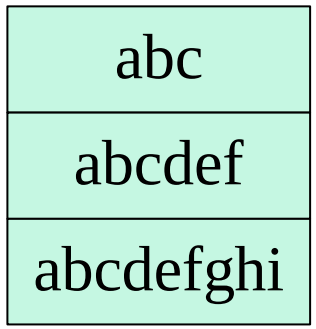
2. 可变字符串StringBuffer

- String创建的字符串是不可变的

String str = “abc”;

str += “def”;

str += “ghi”;



在内存中创新新的值而不是修改原来的值！

耗费大量内存资源！

可变字符串类 **StringBuffer**

StringBuffer

- StringBuffer类是线程安全的可变字符序列，它是一个类似于String的字符串缓冲区，其实体容量会随着存放的字符串增加而自动增加
- 创建StringBuffer类对象的常用方法
  - StringBuffer s=new StringBuffer(); //创建一个空的StringBuffer类的对象
    StringBuffer s=new StringBuffer("abc"); //用一个已存在的字符串常量来创建StringBuffer类对象

StringBuffer str = new StringBuffer("32");

初始字符序列值为32
内存所在空间：16个字符（默认容量）

StringBuffer strbuf = new StringBuffer(32);

初始字符序列容量为32个字符
内存所在空间：32个字符

StringBuffer

- 常用方法

StringBuffer	append(String str)
StringBuffer	**delete**(int start, int end)
StringBuffer	**insert**(int pos, String str)
StringBuffer	**replace**(int start, int end, String str)
StringBuffer	reverse()
void	**setCharAt**(int index, char ch)

例：StringBuffer sbf = new StringBuffer(“012345”);
sbf.setCharAt(3, ‘A’);

StringBuffer

- StringBuffer sbf = new StringBuffer("同一个世界，同一个梦想");
- sbf = sbf.reverse();

- StringBuffer sbf = new StringBuffer("天行健，君子以自强不息";
- sbf = sbf.delete(4, 7);

```
天行健，自强不息
Process completed.
```

StringBuffer

- 与String类似的方法
  - StringBuffer = new StringBuffer(“abcdefg”) ;
  - int length = sbf.length();
  - char c = sbf.charAt(5);
  - int index = sbf.indexOf(“def”);
  - String substr = sbf.substring(0,2);
  - StringBuffer tmp = sbf.replace(2,5, “1234”);

StringBuilder

- StringBuilder类也代表可变字符串对象。StringBuilder和StringBuffer基本相似，两个类的构造器和方法也基本相同。不同的是：StringBuffer是线程安全的，而StringBuilder则没有实现线程安全功能，所以性能略高。
- 由于 StringBuilder 相较于 StringBuffer 有速度优势，所以多数情况下（追求速度）建议使用 StringBuilder 类。然而在应用程序要求线程安全的情况下，则必须使用StringBuffer。

3. 包装类

- 为了将基本数据类型视为对象包装出来，并提供相关的操作方法。java为每个基本类型都提供了包装类：
  - Double(double num);
  - Float(float f);
  - Byte(byte m);
  - Short(short s);
  - Integer(int num);
  - Long(long num);
  - Character(char c);

在执行变量类型转换的时候，会大量使用包装类

方　　法	返回值类型	返回值
Boolean.getBoolean(“false”)	boolean	false
Integer.parseInt(“123”)	int	123
Long.parseLong(“375”)	long	375
Float.parseFloat(“345.23”)	float	345.23
Double.parseDouble(“67892.34”)	double	67892.34

包装类中的一些共同方法

- 带有基本值参数并创建包装类对象的构造函数，如可以利用Integer包装类创建对象，**Integer obj=new Integer(145)**;
- 带有字符串参数并创建包装类对象的构造函数，如 **new Integer("-45.36")**;
- 转换为字符串的**toString()**方法,如 obj.toString().
- 对同一个类的两个对象进行比较的**equals()**方法,如obj1.eauqls(obj2);
- 生成哈稀表代码的**hashCode**方法,如obj.hashCode();
- 将字符串转换为基本值的 **parseType方法**,如Integer.parseInt(args[0]);
- 可生成对象基本值的**typeValue方法**,如obj.intValue();

自动装箱和拆箱

- JDK1.5版本以上均有基本数据类型和相应的对象类型之间相互转换的功能，称之为自动装箱和拆箱
- **自动装箱**：把一个基本数据类型的值直接赋值给基本数据类型相对应的类的实例，如：
  - Integer num = 5;
  - 装箱过程为：Integer num = new Integer(5);
- **自动拆箱**：把基本数据类型相对应的类的实例直接赋给相应的基本数据类型变量，或把基本数据类型相对应的类的实例当做相应的基本数据类型来使用，如：
  - **int x = num**;　　相当于 int x = num.intValue()
  - int x = num + num;相当于 int x = num.intValue() + num.intValue();

Character类

- Character是char的包装类
  - Character ch = new Character('a'); 或 Character ch = 'a'; //自动装箱
- 一些常用方法
  - isLetter(char ch)：检查字符是否为字母。
  - isDigit(char ch)：检查字符是否为数字。
  - isWhitespace(char ch)：检查字符是否为空白字符，如空格、制表符、换行符等
  - isUpperCase(char ch)：检查字符是否为大写字母。
  - isLowerCase(char ch)：检查字符是否为小写字母。
  - toUpperCase(char ch)：将字符转换为大写形式。
  - toLowerCase(char ch)：将字符转换为小写形式。
  - toString(char ch)：将字符转换为字符串。
  - isLetterOrDigit(char ch)：检查字符是否为字母或数字。
  - isJavaIdentifierStart(char ch)：检查字符是否可以作为 Java 标识符的起始字符。
    isJavaIdentifierPart(char ch)：检查字符是否可以作为 Java 标识符的部分字符。
  - digit(char ch, int radix)：将字符按照指定进制转换为对应的整数值。
  - compare(char x, char y)：比较两个字符的大小。

4. 枚举类

- 枚举类(Java.lang.Enum)一般用来表示一组相同类型的常量
- 枚举类型的定义
  - enum WeekDay
  - {
  - SUN, MON, TUE, WED, THU, FRI, SAT
  - }
- 枚举类型值的访问
  - WeekDay day = WeekDay.MON;
  - WeekDay a[] = WeekDay.values();

示例1

```
public class EnumTest {
    public enum MyColor {RED, GREEN, BLUE};
    public static void main(String [] args)  {
        MyColor m = MyColor.RED;
        switch(m)  {
            case RED:  System.out.println("red");  break;
            case GREEN:  System.out.println("green");  break;
            default: System.out.println("default");
    }
    System.out.println(m);
  }
}
```

EnumTest.java

实战

- 实战：

定义某水果的枚举类型，包含几种常用的水果，然后用for、switch语句访问并打印水果中部分水果的价格

Example4_03

5. java.io.File类常用方法

- **createNewFile()**

  当且仅当不存在具有此抽象路径名指定名称的文件时，不可分地创建一个新的空文件。

- **delete()**

  删除此抽象路径名表示的文件或目录。

- **deleteOnExit()**

  在虚拟机终止时，请求删除此抽象路径名表示的文件或目录

- **createTempFile()**

  在默认临时文件目录中创建一个空文件，使用给定前缀和后缀生成其名称。

java.io.File类常用方法

- **listFiles()**

  返回一个抽象路径名数组，这些路径名表示此抽象路径名表示的目录中的文件

- **mkdir()**

  创建此抽象路径名指定的目录

- **list()**

  返回一个字符串数组，这些字符串指定此抽象路径名表示的目录中的文件和目录

- **getName()**

  返回由此抽象路径名表示的文件或目录的名称。

- **exists()**

  测试此抽象路径名表示的文件或目录是否存在。

访问本地文件 - File

- 使用Java.io.File类， 操纵本机文件夹和文件
- 创建文件夹、多级文件夹、创建文件
- “列表显示指定路径下所有子文件夹和文件”
- “删除指定路径下所有子文件夹和文件”

TestFileClass.java

6. 产生随机数

- java.lang.math类的random方法
  - 返回带正号的 double 值，该值大于等于 0.0 且小于 1.0。返回值是一个伪随机选择的数，在该范围内（近似）均匀分布

- java.util.Random类
  - **Random**()
    创建一个新的随机数生成器
  - **Random**(long seed)
    使用单个 long 种子创建一个新的随机数生成器
  - **nextInt**()、 **nextBoolean**()、 **nextDouble**() 等生成随机数

产生随机数

```
//方法一
double a = Math.random();
double b = Math.random();
System.out.println("a = " + a);
System.out.println("b = " + b);
//返回0~2间整数
for (int i=0; i<10; i++)
{
    int c =(int)(Math.floor((Math.random()*10)%3)); //返回最接近整数
    System.out.println(c);
}
```

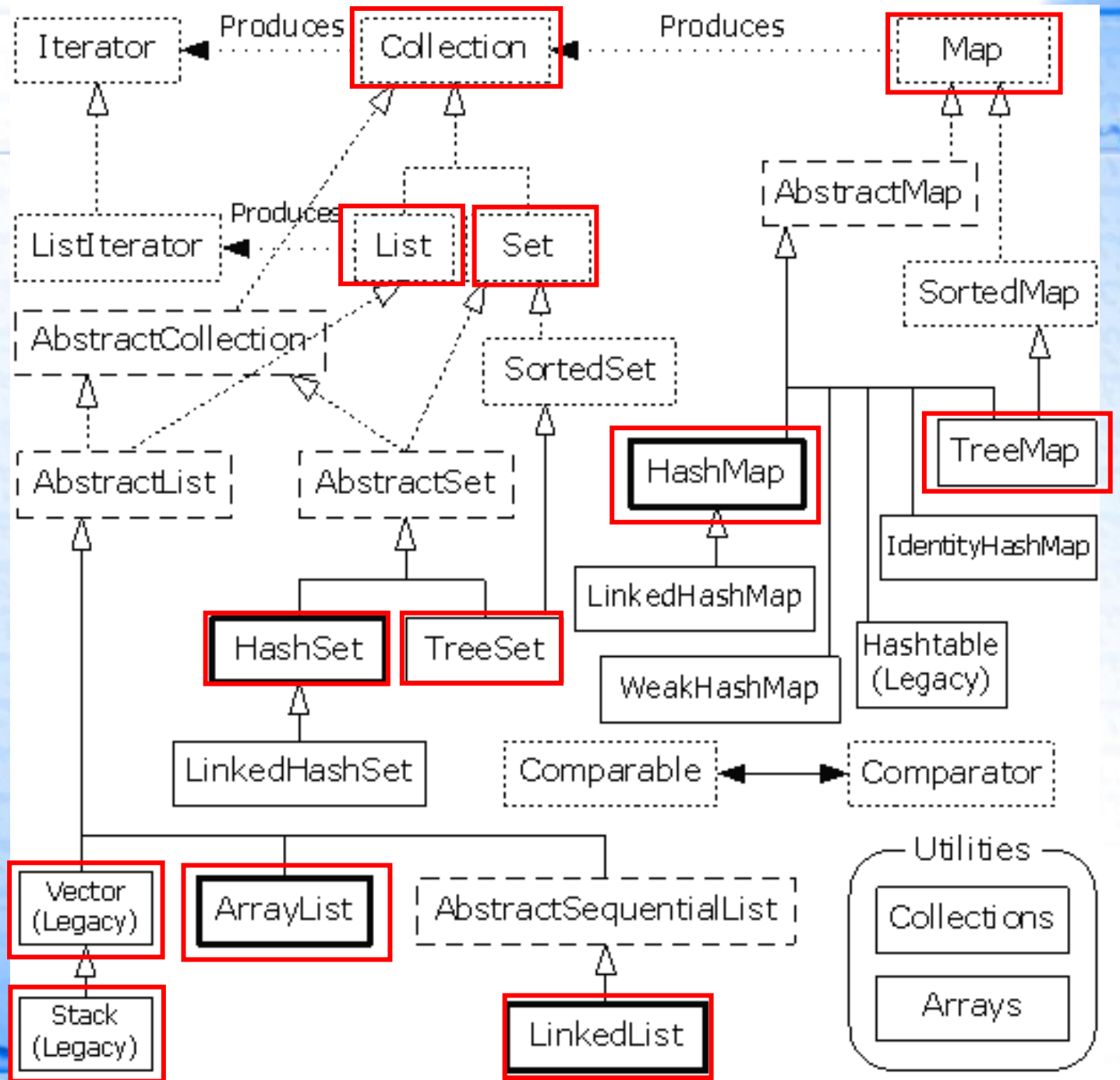
产生随机数

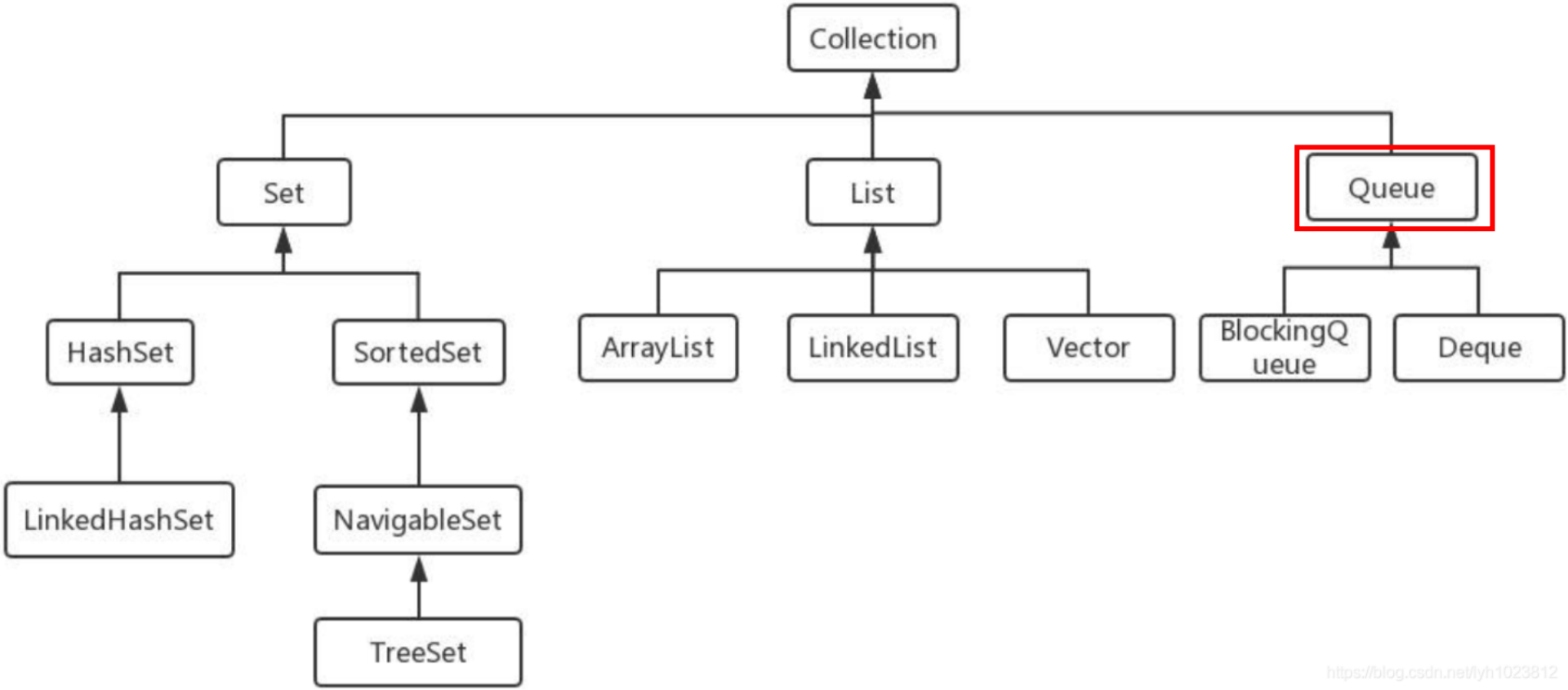
```
//方法二
//无种子，返回值不定
java.util.Random r1=new java.util.Random();
for(int i=0;i<5;i++){
System.out.println("无种： "+ r1.nextInt());
}
//有种子，返回值一定
java.util.Random r2=new java.util.Random(10);
for(int i=0;i<5;i++){
System.out.println("有种： "+ r2.nextInt());
}
```

getRandomNumber.java

7. 集合类

- 在Java.util包中，Java的集合类主要有几种
  - List结构的集合类
    - **ArrayList**，LinkedList，**Vector**，Stack
  - Map结构的集合类
    - HashMap，Hashtable
  - Set结构的集合类
    - HashSet，TreeSet
  - Queue结构的集合
    - Queue接口


Iterator
Produces
Collection
Produces
Map
ListIterator
Produces
List
Set
AbstractMap
SortedMap
AbstractCollection
SortedSet
AbstractList
AbstractSet
HashMap
TreeMap
IdentityHashMap
LinkedHashMap
HashSet
TreeSet
WeakHashMap
Hashtable
(Legacy)
LinkedHashSet
Comparable
Comparator
Utilities
Collections
Arrays
Vector
(Legacy)
ArrayList
AbstractSequentialList
Stack
(Legacy)
LinkedList


Collection
Set
List
Queue
HashSet
SortedSet
ArrayList
LinkedList
Vector
BlockingQueue
Deque
LinkedHashSet
NavigableSet
TreeSet

集合类介绍

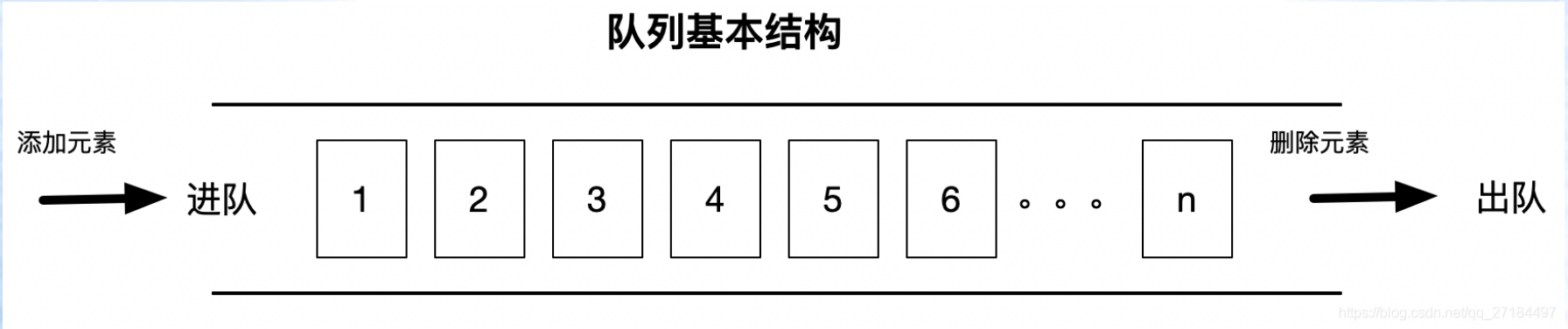
- List、Set、Queue接口都继承于Collection接口
- List
  - 一组有序的对象集合（以元素插入的次序来放置元素，不会重新排列）
  - 允许重复存储对象
  - 功能和数组很相似，可以通过索引来访问List中的元素（get(int index)方法）
  - 内部实现是链表，它适合于需要频繁进行插入和删除的操作，随机访问速度快

集合类介绍

- Set
  - 表示数学中的集合
  - 是无序的，不允许重复存储元素
  - TreeSet是有序的（用二叉树进行排序）
- Map
  - 存储一组成对的对象，即键-值对（Key-value pair）；Key和value之间存在某种映射关系
  - Map中不能有重复的Key，Key无序、唯一；value不要求有序、允许重复。

集合类介绍

- Queue
  - 队列是一种特殊的线性表
  - 遵循先入先出、后入后出的基本原则，一般来说，它只允许在表的前端进行删除操作，而在表的后端进行插入操作

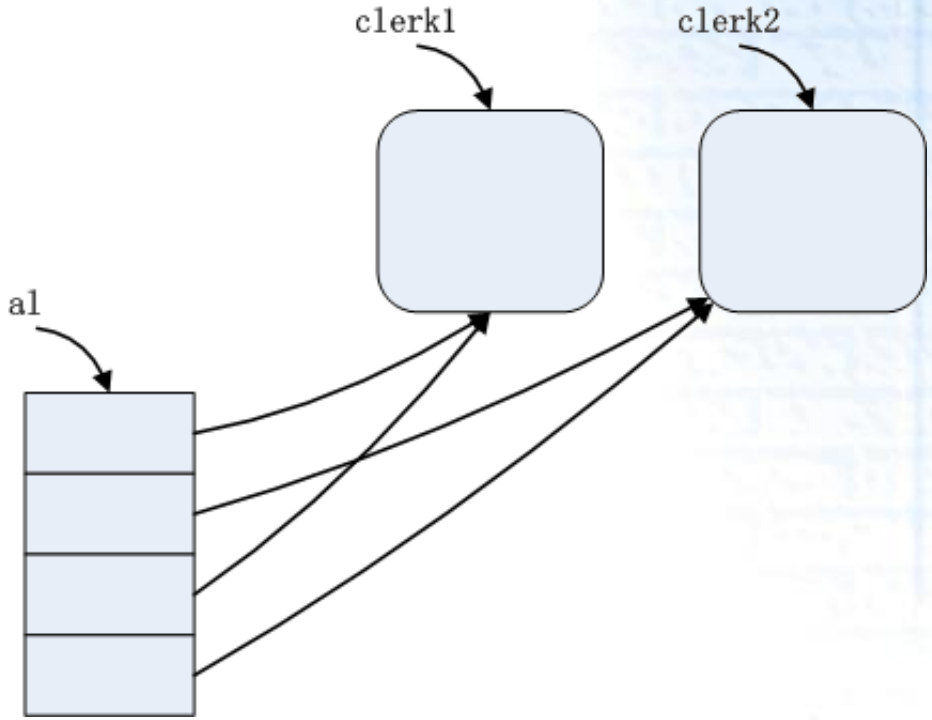


ArrayList

- 添加元素
  - boolean add(E e); 将指定的元素添加到此列表的尾部
  - void add (int index, E element); 将指定的元素插入此列表中的指定位置
- 访问元素
  - get (int index) ; 访问此列表中指定位置上的元素
- 设置元素
  - set(int index, E element); 设置指定位置的元素
- 删除元素
  - remove(int index); 移除指定位置上的元素
  - remove(Object o) ; 移除首次出现的指定元素（如果存在）
  - clear();移除此列表中的所有元素

ArrayList

- 获取大小
  - size(); 获取ArrayList 的大小
- ArrayList中的元素可以重复



实 战

- 创建职工类（包含职工的编号、姓名、薪水信息）的ArrayList结构
- 在ArrayList结构中加入职工类的对象，并进行访问
- 删除ArrayList结构中的职工类对象，并查看删除后的结果

Example01.java

Vector

- Vector是基于数组实现的。其内部包含一个存储元素的数组，所有的元素都被存储在这个数组中。这种基于数组的实现方式使得Vector的访问速度非常快，因为它只需要简单的访问数组的索引即可。
- 与传统的数组不同，Vector可以根据需要动态地增加或减小其大小。当需要增加或减少元素的数量时，Vector会自动调整数组的大小。
- Vector是线程安全的。这意味着它可以在多线程环境下安全地访问和操作，而不必担心数据竞争和不一致的问题。为了实现同步，Vector使用了synchronized关键字，这可能会导致性能下降。

Vector

- 访问元素
  - Vector支持通过索引访问元素。可以使用get(int index)方法来获取指定索引处的元素，或者使用elementAt(int index)方法来获取指定索引处的元素。
- 插入和删除元素
  - Vector支持在指定索引处插入和删除元素。可以使用insertElementAt(Object obj, int index)方法在指定索引处插入元素，或者使用removeElementAt(int index)方法删除指定索引处的元素。
- 克隆
  - Vector支持克隆操作，即创建一个与原始Vector具有相同元素但副本的新Vector。可以使用clone()方法来创建一个克隆对象。

TestVector.java

ArrayList 和 Vector 比较

- 均采用数组方式存储数据，都允许直接序号索引元素
- 区别
  - 同步性：Vector是同步的，即线程安全的。多线程环境下，Vector可以保证数据的安全性，但这也影响力Vector的性能，因为同步操作会导致一定的时间开销。如果在单线程环境下使用集合，建议使用ArrayList。ArrayList是异步的，因此其中的对象并不是线程安全的。
  - 数据增长：从内部实现机制来讲ArrayList和Vector都使用数组（Array）来控制集合中的对象。当需要增加元素时，如果元素的数目超出了内部数组目前的长度，它们都需要扩展，Vector默认情况下自动增长原来一倍的数组长度，ArrayList是原来的50%。所以，如果要在集合中保存大量的数据，那么使用Vector有一些优势。

迭代器Iterator

- 迭代器是一个“轻量级”的对象，用来遍历集合中的元素

```
public interface Iterator
{
    boolean hasNext();   //检查集合内是否还有未访问的元素
    object next();       //返回集合内的下一个元素
    void remove();       //删除迭代器所指向的当前元素
}
```

- 编写访问集合的更通用的方法
- 示例：利用Iterator遍历HashMap

Example05.java

用迭代器来访问向量

```
//Iterator的使用
import java.util.*;
public class example09_09 {
        public static void main(String[] args)  {
                Vector v = new Vector();
                v.add("A");
                v.add("B");
                v.add("C");
                for(Iterator i= v.iterator(); i.hasNext(); )  {
                        System.out.println(i. next()  + "\t");  }
        }  }
```

Example07.java

ListIterator

- ListIterator是专门为List提供的一个功能更为强大的迭代器，继承自Iterator

方法摘要	
void	**add**(E e) 将指定的元素插入列表（可选操作）。
boolean	**hasNext**() 以正向遍历列表时，如果列表迭代器有多个元素，则返回 true（换句话说，如果 next 返回一个元素而不是抛出异常，则返回 true）。
boolean	**hasPrevious**() 如果以逆向遍历列表，列表迭代器有多个元素，则返回 true。
E	**next**() 返回列表中的下一个元素。
int	**nextIndex**() 返回对 next 的后续调用所返回元素的索引。
E	**previous**() 返回列表中的前一个元素。
int	**previousIndex**() 返回对 previous 的后续调用所返回元素的索引。
void	**remove**() 从列表中移除由 next 或 previous 返回的最后一个元素（可选操作）。
void	**set**(E e) 用指定元素替换 next 或 previous 返回的最后一个元素（可选操作）。

ListIterator

- 和Iterator相比较，ListItereator的功能有所丰富，体现在：
  - Iterator只能向后遍历，而ListIterator可以双向遍历
  - 增强了索引操作
    - nextIndex()、previousIndex()返回索引值
    - ListIterator 可以直接定位到List的最后一个元素。
  - 提供了set()和get()方法，可以在遍历时安全地修改集合

ListIterator

- 利用ListIterator进行遍历

```
ArrayList al = new ArrayList();


for(ListIterator liter = al.listIterator(); liter.hasNext();)
{     System.out.println (liter.next());     }
```

或：

```
for(ListIterator liter = al.listIterator(al.size());
   liter.hasPrevious();)
{      System.out.println (liter.previous());    }
```

Example08.java

泛 型

- 泛型的本质是参数化类型，也就是说所操作的数据类型被指定为一个参数。这种参数类型可以用在类、接口和方法的创建中，分别成为泛型类、泛型接口、泛型方法。
- Java中引入泛型的好处是安全简单
- 在Java SE 1.5之前，没有泛型的情况下，通过对类型Object的引用来实现参数的“任意化”，“任意化”带来的缺点是要做显示的强制类型转换，而这种转换是要求开发者对实际参数类型可以预知的情况下进行的，对于强制类型转换错误的情况，编译器可能不提示错误，在运行的时候才出现异常，这是一个安全隐患。
- 泛型的好处是在编译的时候检查类型安全，并且所有的强制转换都是自动和隐式的，提高代码的重用率。

- 例如：
  - ArrayList al = new ArrayList();
  - Dog dog1 = new Dog();
  - al.add(dog1);
  - Cat cat1 = new Cat();
  - al.add(cat1);
  - Dog tempDog = **(Dog)** al.get(0);
  - Cat tempCat = **(Cat)** al.get(1);
- 改为：
  - ArrayList<**Dog**> al = new ArrayList<**Dog**>();
  - Dog temp = al.get(0);　　//不需要强制转换

Example09.java

泛 型

- 优点
  - 类型安全
  - 向后兼容
  - 层次清晰
  - 性能较高，用泛型Java编写的代码可以为Java编译器和虚拟机带来更多的类型信息，这些信息对Java程序做进一步的优化提供条件。

书上的例子

8. 格式化输出Printf

- Java 格式化输出 用printf
- 字符串格式化
  - %s：输出字符串
  - System.out.printf("%s", "hello world"); // 输出 "hello world"
- 整数格式化
  - %d：输出十进制整数
  - %o：输出八进制整数
  - %x：输出十六进制整数
  - %X：输出大写十六进制整数

System.out.printf(“%d %o %x %X”, 10, 10, 10, 10); // 输出 “10 12 a A”

Printf

- 浮点数格式化
  - %f：输出浮点数
  - %e：输出科学计数法表示的浮点数（小写e）
  - %E：输出科学计数法表示的浮点数（大写E）
  - %.nf：控制小数点后的位数，n为数
  - 例：

System.out.printf("%.2f %e %E", 3.1415926, 3.1415926, 3.1415926); // 输出 "3.14 3.141593e+00 3.141593E+00”

- 布尔值格式化
  - %b：输出布尔值
- 字符格式化
  - %c：输出字符

……

Class is Dismissed!

